

SCALABLE-C

Production-Ready Horizontally Scalable WebSocket Chat System

Project Name	Scalable-C (WebSockets Monorepo)
Primary Architectures	Next.js, Socket.IO, Redis Pub/Sub, Turborepo
Documentation Version	v1.0.0 (Production Release)
Author	Karan Suthar
Date Generated	July 6, 2026

Table of Contents

1	Executive Summary & Architectural Problem Statement	Page 3
2	System Architecture & Redis Pub/Sub Synchronization	Page 4
3	Technology Stack Reference Table	Page 5
4	Monorepo Project Layout & File Structure	Page 6
5	Backend System Implementation Walkthrough (Node.js & ioredis)	Page 7
6	Frontend Context Provider & UI Implementation (Next.js)	Page 8
7	Setup, Configuration & Step-by-Step Installation Guide	Page 9
8	Conclusion & Production Scalability Roadmaps	Page 10

1. Executive Summary & Overview

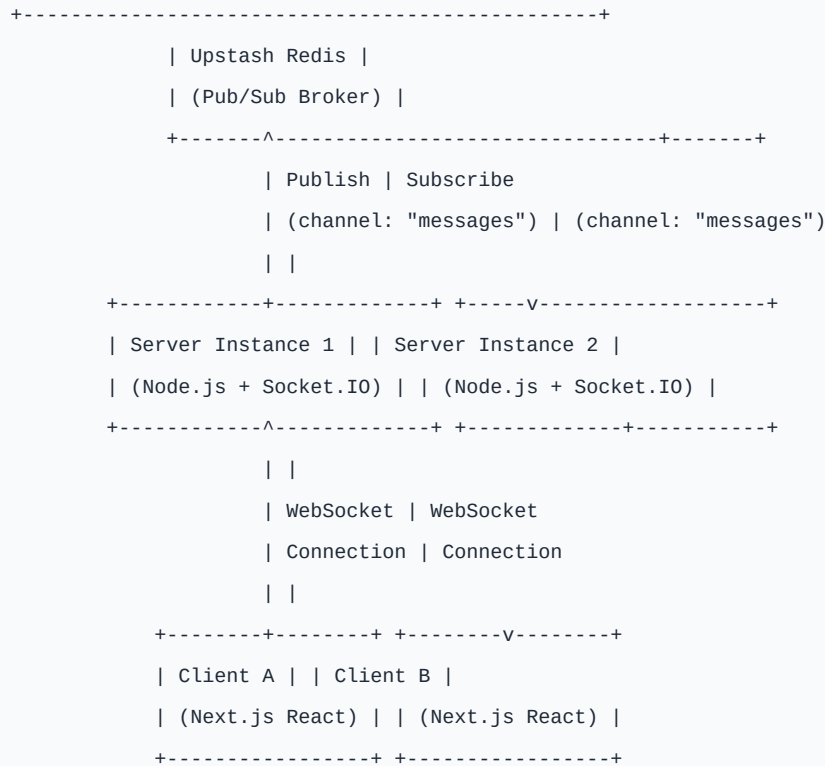
Modern real-time applications rely on persistent WebSocket connections to deliver instant updates. However, as user traffic grows, a single WebSocket server instance becomes a bottleneck, capped by CPU, memory, and maximum file descriptors. Scaling WebSocket servers horizontally (adding more instances) introduces a critical synchronization problem: WebSockets are inherently **stateful**. A client establishes a persistent TCP link to one specific server instance. If Client A connects to **Server Instance 1** and Client B connects to **Server Instance 2**, they cannot communicate directly. When Client A sends a message, Server 1 receives it but does not know Client B exists, resulting in disconnected message silos.

Scalable-C solves this architectural challenge using a **Redis-backed Publish/Subscribe (Pub/Sub)** broker. Instead of servers trying to talk to each other directly or routing through a bottleneck database, every server instance connects to a shared, high-performance Redis channel. When a server instance receives an event from any local WebSocket client, it publishes the event to Redis. Every active server instance subscribes to that channel, receives the broadcast instantly, and distributes it to its own connected local clients. This decouples connections from event routing, allowing the server tier to scale out indefinitely.

Key Architectural Advantage: By utilizing serverless Upstash Redis, the network broker tier is kept fully serverless and highly performant. It supports instant message delivery across instances worldwide with minimal latency, avoiding the overhead of managing self-hosted Redis setups.

2. System Architecture & Message Flow

The diagram below demonstrates how Redis Pub/Sub bridges the communication gap between multiple decoupled server instances:



Detailed Step-by-Step Message Flow:

- 1. Client-to-Server Event:** Client A sends a chat message via WebSocket. The event is emitted as 'event:message' and lands on Server Instance 1.
- 2. Redis Publish:** Server Instance 1 parses the event and publishes the payload stringified as JSON to the Redis channel named 'messages'.
- 3. Redis Broadcast:** Redis distributes the payload to all server instances currently subscribed to the 'messages' channel (Server 1 and Server 2).
- 4. Local Socket Emit:** Upon receiving the message from Redis, Server Instance 2 emits the message locally using `io.emit('message')` to all of its connected WebSocket clients, including Client B.
- 5. Client Render:** Client B's socket handler catches the incoming message and updates the local React state, presenting the message immediately in the chat interface.

3. Technology Stack Reference

The following table highlights the technologies chosen, their specific roles in the project, and the engineering rationale behind them:

Technology / Layer	General Use & Why It's Used	Specific Use In This Project
Next.js (React) Frontend Framework	Next.js is an industry-standard React framework used generally for building highly optimized, SEO-friendly web applications. Developers choose it for its built-in routing, server-side rendering (SSR), and excellent developer experience.	In this project, Next.js powers the client-side user interface (the web app). It serves the chat UI and manages the React state for incoming and outgoing messages while initializing the Socket.IO client.
Socket.IO Real-Time Engine	Socket.IO is a widely-used library that enables low-latency, bidirectional, and event-based communication. It is preferred over raw WebSockets because it provides auto-reconnection, room management, and HTTP long-polling fallbacks.	Used here on both frontend and backend to establish the persistent connection. It handles the actual transmission of chat messages ('event:message') between the user's browser and the Node.js server.
Node.js & Express Backend Framework	Node.js is a JavaScript runtime ideal for I/O-intensive tasks. Express is a minimalist web framework for Node.js. They are used together to quickly build scalable network applications and REST APIs.	Serves as the foundation for the server app. Express sets up the HTTP server, which is then hijacked by Socket.IO to upgrade incoming connections to WebSockets. It also manages connections to Upstash Redis.
Upstash Redis Pub/Sub Engine	Redis is an in-memory data structure store, used generally as a database, cache, and message broker. Upstash provides a serverless Redis offering that auto-scales and bills per request, eliminating infrastructure management.	Acts as the critical message broker that enables horizontal scaling. When a message hits one server, it publishes to Upstash Redis, which instantly broadcasts it to all other connected server instances.
Turborepo Monorepo Tool	Turborepo is a high-performance build system for JavaScript/TypeScript codebases. It is used generally to manage complex 'monorepos' (multiple projects in one repository) by caching build outputs and running scripts in parallel.	Orchestrates the entire repository. It allows a single yarn dev command to simultaneously start the Next.js frontend and Node.js backend, and shares configurations (like ESLint and TypeScript) seamlessly between them.

TypeScript Language	TypeScript is a strongly typed superset of JavaScript. It is used to catch errors at compile-time rather than run-time, significantly improving code quality, developer tooling, and maintainability in large codebases.	Provides end-to-end type safety. By using TypeScript across both the web and server apps, we ensure that data structures (like the message payloads sent over WebSockets) remain strictly consistent across the network boundary.
-------------------------------	--	---

Monorepo Rationale: The monorepo structure makes sharing configurations (such as TypeScript, ESLint, and shared React components) extremely simple. It enables a clean separation of concerns while keeping both frontend and backend within a single git history.

4. Monorepo Project Layout & File Structure

Below is a structural tree of the Turborepo workspace. It shows the division of work into apps and package utilities:

```
scalable-c/
├── apps/
│   ├── web/ # Next.js Frontend Client
│   │   └── app/
│   │       ├── layout.tsx # Main HTML root template & font loading
│   │       ├── globals.css # Core global resets
│   │       ├── page.tsx # Chat UI page component with inputs & logs
│   │       └── context/
│   │           ├── SocketProvider.tsx # Client WebSocket manager context provider
│   │           └── package.json
│   └── server/ # Node.js Server Backend
│       ├── src/
│       │   ├── index.ts # Server entry point initializes socket & server
│       │   └── services/
│       │       ├── socket.ts # Socket.IO & Redis pub/sub broker bindings
│       │       └── package.json
│       └── packages/ # Shared workspace utility modules
│           ├── ui/ # Shared global UI design components
│           ├── eslint-config/ # Centralized lint rules
│           ├── typescript-config/ # Centralized strict config for compiler
│           ├── turbo.json # Build pipelines orchestrator
│           └── package.json # Monorepo workspaces root config
```

Configuration Files Breakdown:

- **turbo.json**: Defines cache rules for dev, build, lint, and type-check steps. If package source code hasn't changed, Turborepo will retrieve cached builds, saving developers hours of rebuild time.
- **package.json (Root)**: Configures workspaces array. Next.js and backend apps can install shared configurations using standard workspace aliases like 'packages/*'.
- **packages/typescript-config**: Standardizes compiler configuration to ensure rigorous check constraints are unified between front and back-ends.

5. Backend Implementation Walkthrough

The backend starts a HTTP server and connects to the Upstash Redis cluster. It maintains separate Redis connections for Publishing and Subscribing, which is a required specification in Redis architecture.

■ apps/server/src/index.ts

```
import http from "http";
import SocketService from "../services/socket.js";

async function init() {
  const socketService = new SocketService();

  const httpServer = http.createServer();
  const PORT = process.env.PORT ? process.env.PORT : 8000;

  socketService.io.attach(httpServer);

  httpServer.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`)
    socketService.intListeners();
  });
}

init();
```

■ apps/server/src/services/socket.ts

```
import { Server } from "socket.io";
import { Redis } from "ioredis";

const pub = new Redis({
  host: '<UPSTASH_REDIS_HOST>',
  port: 6379,
  username: 'default',
  password: '<UPSTASH_REDIS_PASSWORD>',
  tls: {} // REQUIRED for Upstash!
});

const sub = new Redis({
  host: '<UPSTASH_REDIS_HOST>',
  port: 6379,
  username: 'default',
  password: '<UPSTASH_REDIS_PASSWORD>',
```



```
    tls: {} // REQUIRED for Upstash!
  });

class SocketService {
  private _io: Server;

  constructor() {
    this._io = new Server({
      cors: {
        allowedHeaders: ["*"],
        origin: "*" // Fixed this CORS issue! It needs to be a single string for wildcard.
      }
    });
    sub.subscribe("messages");
  }

  public intListeners() {
    const io = this._io;
    console.log("Init Socket Listeners...");

    io.on("connect", (socket) => {
      console.log("User connected", socket.id);

      socket.on("event:message", async ({ message }: { message: string }) => {
        console.log("New Message received", message);
        await pub.publish("messages", JSON.stringify({ message }));
      })
    })

    sub.on("message", (channel, message) => {
      if (channel === "messages") {
        console.log("Message received from Redis", message);
        this._io.emit("message", message);
      }
    })
  }

  get io() {
    return this._io;
  }
}

export default SocketService;
```

Backend Architectural Analysis:

- **Dual Redis Client Pattern:** Redis connection handles blocking subscription command. A separate publishing client is required to broadcast outgoing messages. We instantiate two distinct clients: pub and sub.
- **Event Mapping:** We map incoming WS events (event:message) to out-bound Redis publications. When Redis receives it, it triggers the callback sub.on('message') which instantly calls io.emit('message') to push it to the frontend client.
- **CORS Configuration:** Set to allow all origins and headers, eliminating cross-origin errors commonly faced when starting frontend/backend on separate local ports (3000 vs 8000).

6. Frontend Client Walkthrough

The React application wraps user routes inside a `SocketProvider` context. This context establishes a single persistent connection to the server when the client is loaded, and handles the local list of incoming and outgoing chat logs.

■ `apps/web/context/SocketProvider.tsx`

```
'use client';

import React, { useCallback, useEffect, useContext, useState } from "react";
import { io, Socket } from "socket.io-client";

interface SocketProviderProps {
  children?: React.ReactNode;
}

interface ISocketContext {
  sendMessage: (msg: string) => void;
  messages: {text: string, type: 'sent' | 'received'}[];
}

const SocketContext = React.createContext<ISocketContext | null>(null);

export const useSocket = () => {
  const state = useContext(SocketContext);
  if(!state){
    throw new Error("state is undefined");
  }
  return state;
}

export const SocketProvider: React.FC<SocketProviderProps> = ({ children }) => {
  const [socket, setSocket] = useState<Socket | null>(null);
  const [messages, setMessages] = useState<{text: string, type: 'sent' | 'received'}[]>([
    { text: "Welcome to the global chatroom!", type: 'received' }
  ]);

  const sendMessage: ISocketContext['sendMessage'] = useCallback((msg: string) => {
    console.log("Send Message", msg);
    if (socket) {
      socket.emit("event:message", { message: msg });
      setMessages(prev => [...prev, { text: msg, type: 'sent' }]); // Show it instantly as sent
    }
  }, [socket])
```

```

const onMessageRecieved = useCallback((msg: string) => {
  console.log("Message Recieved from Server", msg);
  try {
    const parsed = typeof msg === "string" ? JSON.parse(msg) : msg;
    const messageText = parsed.message;

    // To avoid double-showing messages we *just* sent (since the server broadcasts to
    EVERYONE),
    // we'll filter it out if it already exists as the very last 'sent' message.
    setMessages(prev => {
      const lastMsg = prev.length > 0 ? prev[prev.length - 1] : undefined;
      if (lastMsg && lastMsg.text === messageText && lastMsg.type === 'sent') {
        return prev; // We already showed this!
      }
      return [...prev, { text: messageText, type: 'received' }];
    });
  } catch (e) {
    console.error("Could not parse message", e);
  }
}, [])

useEffect(()=>{
  const socket = io("http://localhost:8000");
  socket.on("message", onMessageRecieved);
  setSocket(socket);
  socket.on("connect", () => {
    console.log("Connected to server");
  });
  return () => {
    socket.disconnect();
    socket.off("message", onMessageRecieved);
    setSocket(null);
  };
}, [onMessageRecieved])

return (
  <SocketContext.Provider value={{ sendMessage, messages }}>
    {children}
  </SocketContext.Provider>
)
}

```

■ apps/web/app/page.tsx

```
'use client';
import { useState } from 'react';
import { useSocket } from '../context/SocketProvider';
import styles from './page.module.css';

export default function Page() {
  const { sendMessage, messages } = useSocket();
  const [message, setMessage] = useState('');

  const handleSend = () => {
    if (message.trim()) {
      sendMessage(message);
      setMessage('');
    }
  };

  const handleKeyDown = (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (e.key === 'Enter') handleSend();
  };

  return (
    <div className={styles.container}>
      <div className={styles.chatBox}>
        <div className={styles.header}>
          Real-time Global Chat
        </div>

        <div className={styles.messagesContainer}>
          {messages.map((msg, index) => (
            <div
              key={index}
              className={` ${styles.message} ${msg.type === 'sent' ? styles.sent : styles.received}`}
            >
              {msg.text}
            </div>
          ))}
        </div>

        <div className={styles.inputArea}>
          <input
            type="text"

```

```
        className={styles.chatInput}
        placeholder="Type a message..."
        value={message}
        onChange={(e) => setMessage(e.target.value)}
        onKeyDown={handleKeyDown}
      />
      <button className={styles.sendButton} onClick={handleSend}>
        Send
      </button>
    </div>
  </div>
</div>
);
}
```

Frontend Code Analysis:

- **State Management and Instability Protection:** When sending a message, the client appends it immediately locally as 'sent' to ensure instant user-feedback. Since the server broadcasts the event back to everyone, we use a filtering mechanism in the `onMessageRecieved` handler to prevent duplicates.
- **Context Cleanups:** The `useEffect` block hooks into the component lifecycle to disconnect the socket and unbind listeners upon unmounting, preventing memory leaks.

7. Setup & Installation Guide

Follow these steps to run the monorepo in a local environment:

1. Install Dependencies

Run the root yarn installation command to setup packages for all sub-apps concurrently:

```
$ yarn install
```

2. Configure Environment Variables

Create a file named `.env` in the `apps/server/` directory. Populate it with Upstash Redis credentials, using secure transport protocols (TLS):

■ apps/server/.env

```
UPSTASH_REDIS_URL=your_upstash_redis_url
UPSTASH_REDIS_TOKEN=your_upstash_redis_token
PORT=8080
```

3. Run the Development Server

Execute the Turborepo dev runner. This launches both apps simultaneously. Turborepo routes and labels output lines according to their source project logs:

```
$ yarn dev
```

The local development urls will resolve to:

Component	Address
Next.js Client (UI)	http://localhost:3000
WebSocket Backend Server	http://localhost:8000 (Socket.IO)

8. Conclusion & Production Scalability Roadmaps

This project establishes a solid foundation for building scaling real-time message routers. However, to take this into high-load production environments, the following scalability strategies should be prioritized:

1. Load Balancing and Sticky Sessions

When placing WebSocket instances behind a Load Balancer (like NGINX, AWS ALB, or Cloudflare), you must enable **Sticky Sessions** (session affinity). This is because the Socket.IO handshake requires the client to send subsequent HTTP polling requests to the same server before upgrading to a persistent WebSocket connection. Without session affinity, requests will route to random instances, causing handshake failures.

2. Database Persistence & Offline Queueing

Currently, chat messages are transient — they only exist in memory and in the active stream of Redis. If a user is offline, they miss the message entirely. In production, every incoming message should be persisted to a durable database (e.g. PostgreSQL, MongoDB, or Cassandra). Additionally, an offline notifications worker can check if a target user is offline, queuing push notifications or emails accordingly.

3. Socket.IO Adapter optimization

While we implemented custom ioredis pub/sub logic, Socket.IO provides an official adapter (@socket.io/redis-adapter). For complex room management (e.g., separating users into distinct chatrooms like 'room-101'), switching to the official Redis adapter is recommended. It handles room joining, leaving, and client queries across server boundaries automatically.

4. Security Hardening

- **Authentication:** Wrap connections in authentication middleware (e.g., JWT verifying headers during the initial handshake).
- **TLS/WSS:** Run connections over HTTPS and WSS to protect data in transit.
- **Rate Limiting:** Implement connection rate limiters to prevent DDoS and spam on socket creation endpoints.